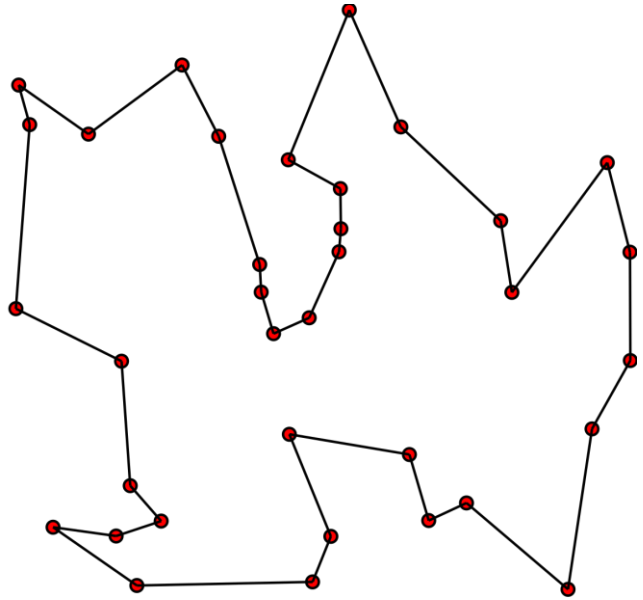
Travelling Salesman Problem

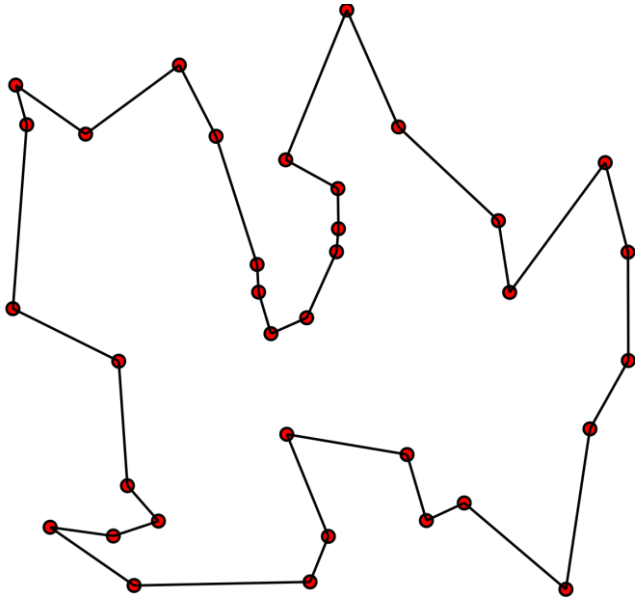


Sensitivity: Internal

The Problem

Given n cities and the cost c_{ij} of travelling between each pair of cities (i, j).

Find the cheapest way of visiting all cities and returning to your starting point.



Sensitivity: Internal

Complexity

Starting point given. How many routes are there?

Let’s say we have 4 cities, look at the following two routes

$$A - B - C - D - A$$

$$A - D - C - B - A$$

Do these cost the same or are they different?

They cost the same, therefore we treat them as the same route.

Sensitivity: Internal

Complexity

From our starting point, how many choices do we have for the next city?

$$3 = 4 - 1$$

For our next city?

$$2 = 3 - 1$$

Thus, we have $3!$ possible routes? Are we done?

No, you can go in either direction

$$A - B - C - D - A$$
$$A - D - C - B - A$$

Divide by 2

$$\frac{3!}{2}$$

Sensitivity: Internal

Complexity

In general, we have to compare $\frac{(n-1)!}{2}$ routes

So, a brute force algorithm will be of order

$$O(n!)$$

That is, the time taken grows roughly like $n!$ as n increases!

Ouch!

Sensitivity: Internal

Some Numbers…

"It would take a laptop 1,000 years to compute the most efficient route between 22 cities."

Washington Post - Feb 5th , 2018

25,545,471,085,854,720,000 undirected tours for $n = 22$.

This is partly right, what does it assume?

It assumes brute force is the only way to compute this!

Sensitivity: Internal

NP-Complete

In its yes/no form (is there a tour of length at most L?) the TSP is in NP: Nondeterministic Polynomial time.

The NP bit means that a proposed solution can be checked in polynomial time, i.e. $O(n^p)$, e.g. $O(n^3)$

The Complete bit means it is among the hardest problems in NP: a polynomial-time algorithm for it would give one for every problem in NP. None is known. Finding the shortest tour is NP-hard.

Sudoku (on $n^2 \times n^2$ grids) is also NP-complete.

Sensitivity: Internal

$P = NP?$

- $1,000,000 Millennium Problem

- An algorithm that solves TSP in polynomial time would win you $1,000,000.

Sensitivity: Internal

Applications

Routing/Timetabling/Scheduling

- Bus Routes
- Service Calls by BT/Sky

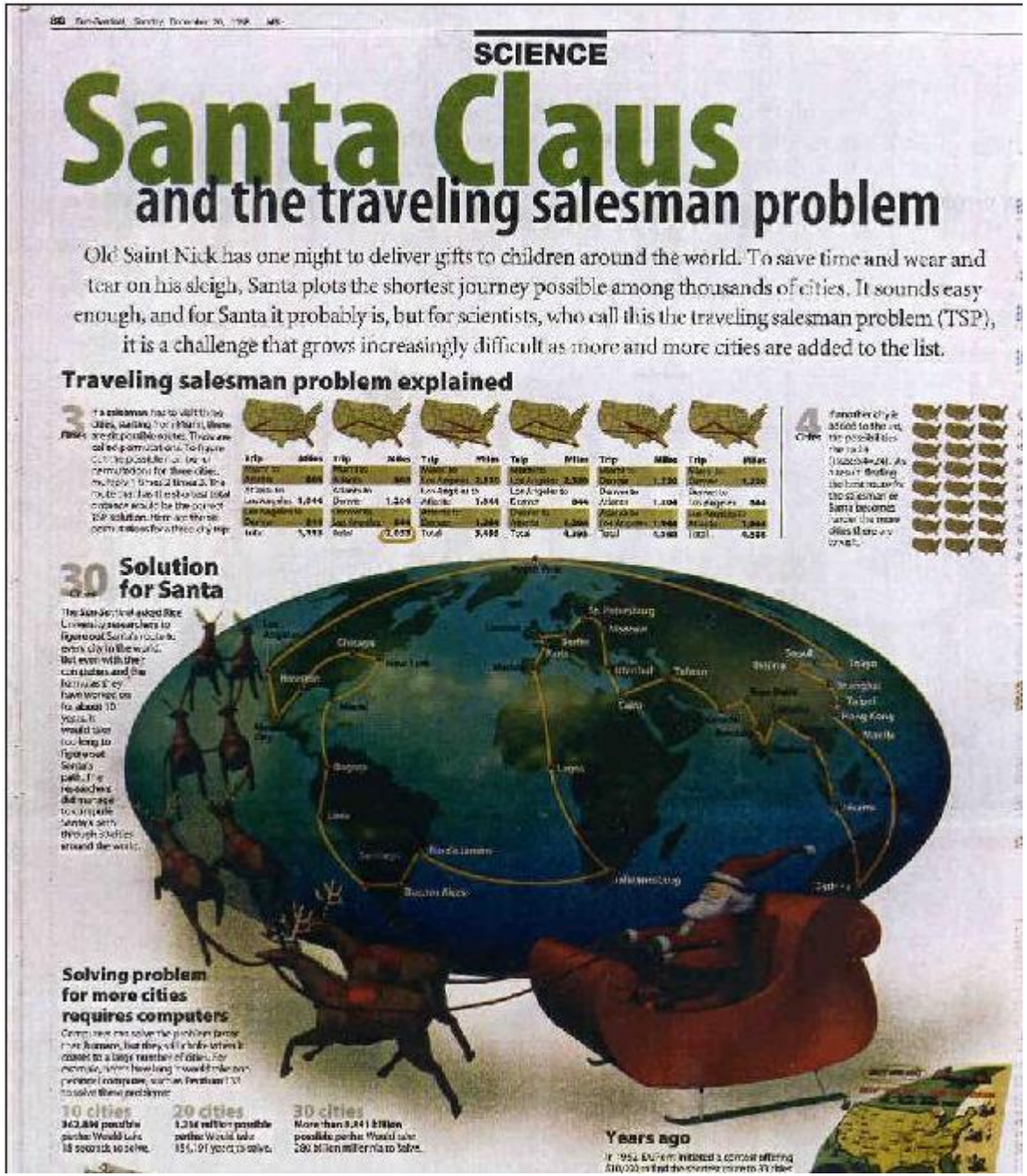
Manufacturing Process

- Drilling the holes in a circuit board, in which order does the machine do this?

Sensitivity: Internal

TSP in the news…

Florida Sun-Sentinel presented the following article on 20 December 1998



SCIENCE

Santa Claus
and the traveling salesman problem

Old Saint Nick has one night to deliver gifts to children around the world. To save time and wear and tear on his sleigh, Santa plots the shortest journey possible among thousands of cities. It sounds easy enough, and for Santa it probably is, but for scientists, who call this the traveling salesman problem (TSP), it is a challenge that grows increasingly difficult as more and more cities are added to the list.

Traveling salesman problem explained

3 Cities

4 Cities

30 Solution for Santa

Solving problem for more cities requires computers

10 cities
363,894 possible paths: Would take 18 seconds to solve.

20 cities
1.216 million possible paths: Would take 155.19 years to solve.

30 cities
More than 8,841 billion possible paths: Would take 280 billion millennia to solve.

Years ago

Sensitivity: Internal

William Cook

University Professor, Combinatorics and Optimization, University of Waterloo

https://www.math.uwaterloo.ca/~bico/

He has written several books and given excellent presentations on the TSP

Well worth a look

Sensitivity: Internal

MILP

Let x_{ij} = 1 if the edge (i, j) connecting city i and city j is used (0 otherwise), and let c_{ij} be the cost of that edge.

Then we could try and model the problem as:

$$\min. \sum_{(i,j)\in E} c_{ij}x_{ij}$$

$$s.t.$$

$$\sum_{j=1:i\neq j}^{n} x_{ij} = 1 \quad \forall i \in \{1, \dots, n\}$$

$$\sum_{j=1:i\neq j}^{n} x_{ji} = 1 \quad \forall i \in \{1, \dots, n\}$$

$$x_{ij} \in \{0,1\}$$

Why is this formulation wrong?

Sensitivity: Internal

MILP

$$\min. \sum_{(i,j)\in E} c_{ij}x_{ij}$$

$$s.t.$$

$$\sum_{j=1:i\neq j}^{n} x_{ij} = 1 \quad \forall i \in \{1,\dots,n\}$$

$$\sum_{j=1:i\neq j}^{n} x_{ji} = 1 \quad \forall i \in \{1,\dots,n\}$$

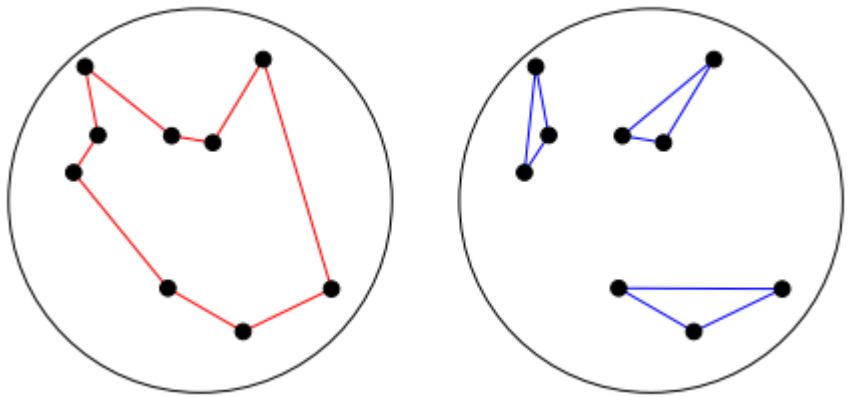
$$x_{ij} \in \{0,1\}$$

Whilst all cities will be visited, it doesn't guarantee a single tour!

We could have lots of subtours that cover all cities.

Sensitivity: Internal

Subtour example



$$\min. \sum_{(i,j)\in E} c_{ij}x_{ij}$$

$$s.t.$$

$$\sum_{j=1:i\neq j}^{n} x_{ij} = 1 \quad \forall i \in \{1, \dots n\}$$

$$\sum_{j=1:i\neq j}^{n} x_{ji} = 1 \quad \forall i \in \{1, \dots n\}$$

$$x_{ij} \in \{0,1\}$$

Both the red (complete tour) and the blue (subtours) have exactly one edge entering and one edge leaving a city!

Sensitivity: Internal

Solution

Introduce dummy variables $u_i \in \mathbb{R} \ \forall i \in \{2, \dots, n\}$ that track the order in which city i is visited.

Then if $x_{ij} = 1$ (that is edge (i, j) is chosen)

- City i is visited before city j
- Hence, we require $u_j > u_i$ which can be written $u_j \geq u_i + 1$

The following constraints do this.

$$u_i - u_j + 1 \leq (n-1)\big(1 - x_{ij}\big) \qquad \forall i, j \in \{2, \dots, n\}: i \neq j$$

Sensitivity: Internal

Why does this work?

The incoming and outgoing constraints mean that for some j

$\boldsymbol{x_{ij} = 1}$

$$u_i - u_j + 1 \leq 0$$

Which is the same as:

$$u_j \geq u_i + 1$$

Hence selecting edge (i, j) means j visited after i.

This means that $\boldsymbol{x_{ji} = 0}$.

Why? Consider if $\boldsymbol{x_{ji} = 1}$, then $u_i \geq u_j + 1$ and $u_j \geq u_i + 1$

This is a contradiction.

Sensitivity: Internal

Why does this work?

We can continue this reasoning.

We know that there must exist an r_2 such that $x_{1r_2} = 1$ (i.e. we leave city 1).

There also must be $x_{r_2r_3} = 1$, and $x_{r_3r_4} = 1$, and $x_{r_4r_5} = 1$, and …

Thus, $u_{r_2} < u_{r_3} < u_{r_4} < u_{r_5} < \cdots < u_{r_n}$.

Hence there will be an increasing sequence of u_i, thus precluding $u_i = u_j$.

Sensitivity: Internal

Why does this work?

Once this sequence is in place.

$$u_{r_2} < u_{r_3} < u_{r_4} < u_{r_5} < \cdots < u_{r_n}$$

Then we have to choose some k such that:
$x_{k1} = 1$ i.e. some edge enters city 1.

However, the only remaining city which doesn't have an edge leaving is r_n.

Thus, $k = r_n$, i.e., $x_{r_n 1} = 1$

This is valid as we did not include city 1 in the MTZ constraint. Hence, we have a tour $1, r_2, r_3, r_4, \ldots, r_n, 1$

Sensitivity: Internal

Miller-Tucker-Zemlin (MTZ) Formulation

$$\min. \sum_{(i,j)\in E} c_{ij}x_{ij}$$

$$s.t.$$

$$\sum_{j=1:i\neq j}^{n} x_{ij} = 1 \qquad \forall i \in \{1,\dots,n\}$$

$$\sum_{j=1:i\neq j}^{n} x_{ji} = 1 \qquad \forall i \in \{1,\dots,n\}$$

$$u_i - u_j + 1 \leq (n-1)\left(1 - x_{ij}\right) \qquad \forall i,j \in \{2,\dots,n\}: i \neq j$$

$$x_{ij} \in \{0,1\}$$
$$u_i \in \mathbb{R}$$

Sensitivity: Internal

Pyomo Code

The demonstration notebook runs in your browser on the course site: Week 09, 2 Lecture Demos, 1 TSP with Pyomo.ipynb.

You can also download it and run it in Google Colab or Anaconda.

Let’s try and solve larger and larger problems…

Sensitivity: Internal

Heuristics

We can search for a **good** sub-optimal solution.

[Traveling Salesman Problem Visualization - YouTube]

Sensitivity: Internal

Tutorial

1. Please use the ipynb notebook to investigate the travelling salesman problem (TSP) - 1 TSP with Pyomo.ipynb on the course site
   - You should try out the problem.
   - Increase the problem size, when does it become unreasonable to solve?
   - Try messing around with the parameters of the genetic algorithm, such as mutation, crossover and population. Can you solve larger TSP problems than the MILP can?
2. I would like you to do some research on the travelling salesman problem.
   - Why is it hard to solve?
   - What is NP-hard?
   - What is combinatorial optimisation?

Sensitivity: Internal

A must watch video

[Math Encounters: "Optimal Tours: The Traveling Salesman Problem" with Bill Cook on August 4, 2021]

Sensitivity: Internal